

Product Guide

Admin Setup & Usage — complete documentation

v1.0.0

June 2026

Contents

PART I — ADMIN SETUP

1. Prerequisites
2. Choose How To Run It
3. Environment Variables (server/.env)
4. Database & Migrations
5. First Admin & Demo Accounts
6. Roles & Permissions
7. Vendor Integrations (Mock → Live)
8. Local LLM Evaluation (Optional)
9. CI & The Security-Score Gate
10. Production Hardening Checklist
11. Troubleshooting

PART II — USAGE GUIDE

1. Sign In
2. Dashboard — Read Your Posture
3. LLM Stack Map — See The Architecture
4. Layer Assessment — Mark Maturity AppSec / Owner
5. Risk Register — Track Threats SOC / AppSec / Owner
6. Control Library & Architecture Builder
7. File Security Gateway — Scan A File SOC / AppSec / Owner
8. Lakera Guard — Inspect A Prompt SOC / AppSec / Owner
9. Model Coverage — Register, Test & Evaluate
10. Integrations
11. Compliance — Frameworks & Reports
12. Report Generator
13. Settings
14. ★ Quick Role Reference

Part I · Admin Setup Guide

1. Prerequisites

- **Node.js 20+** and npm
- **Docker + Docker Compose** — for the one-command full-stack / Postgres path
- *(Optional)* **Ollama** or any OpenAI-compatible local LLM runtime, for live model evaluation

2. Choose How To Run It

Option A — Docker Compose (recommended)

One command brings up **Postgres + API + nginx-served frontend**:

```
docker compose up --build      # → http://localhost:8080
```

The API auto-applies the schema and seeds demo data. Tear down with `docker compose down` (add `-v` to drop the database volume).

Option B — Full Stack, Local Dev (SQLite)

```
# 1) Backend
cd server
npm install
cp .env.example .env
npm run setup      # prisma generate + migrate deploy + seed
npm start          # API on http://localhost:4000

# 2) Frontend (separate terminal)
cd ..
npm install
npm run dev        # http://localhost:5173 (proxies /api → :4000)
```

Option C — Frontend Only (offline demo)

```
npm install && npm run dev      # http://localhost:5173
```

Runs entirely on seed data + in-browser mocks — the header shows **Local mode** and no login is required.

3. Environment Variables (`server/.env`)

VARIABLE	PURPOSE	DEFAULT
<code>DATABASE_URL</code>	Prisma connection string	<code>file:./dev.db</code>
<code>PORT</code>	API port	4000
<code>CORS_ORIGIN</code>	Allowed browser origin(s)	<code>http://localhost:5173</code>
<code>JWT_SECRET</code>	Session signing secret — set in production	ephemeral
<code>NODE_ENV</code>	<code>production</code> blocks demo seeding (§5)	unset
<code>ALLOW_SEED</code>	<code>true</code> to allow seeding in production	unset
<code>CHECKPOINT_TE_API_KEY</code> / <code>_BASE_URL</code>	Check Point Threat Emulation	mock
<code>CHECKPOINT_FILE_SCAN_API_KEY</code> / <code>_BASE_URL</code>	Check Point TE File Scan	mock
<code>LAKERA_GUARD_API_KEY</code> / <code>_BASE_URL</code>	Lakera Guard	mock

Secrets are read at runtime and never hardcoded. In production, supply them via your platform's secret store and terminate TLS in front of the API so the `Secure` session cookies apply.

4. Database & Migrations

- Schema: `server/prisma/schema.prisma` ; committed migrations under `server/prisma/migrations/` .
- Apply: `npm run db:migrate` (`prisma migrate deploy`). Seed: `npm run db:seed` . `npm run setup` does generate + migrate + seed.
- Postgres**: point `DATABASE_URL` at Postgres (Docker path does this); the schema is provider-portable.

5. First Admin & Demo Accounts

Demo accounts (dev/seed only) — all use password `Demo!1234` :

EMAIL	ROLE
<code>owner@imbsys.com</code>	Owner
<code>appsec@imbsys.com</code>	AppSec
<code>soc@imbsys.com</code>	SOC
<code>viewer@imbsys.com</code>	Viewer

Production: seeding is blocked when `NODE_ENV=production` (unless `ALLOW_SEED=true`). Create your first **Owner** with the bootstrap CLI:

```
cd server
npm run create:admin -- --email you@org.com --name "You" --password 'StrongPass!23'
```

The same command resets an existing user to Owner with a new password. Manage everyone else from **Settings** → **User Management**.

6. Roles & Permissions

ROLE	CAN DO
Owner	Everything, incl. integration toggles & user management
AppSec	Edit controls, risks, models; run file scans & prompt inspections
SOC	Edit risks/incidents; run scans & inspections; read-only elsewhere
Viewer	Read-only

Enforced server-side (`requirePerm` → 401/403) and mirrored in the UI.

7. Vendor Integrations (Mock → Live)

Integrations run in **mock mode** until keys are configured (§3). Once a key + base URL are set, the integration flips to **live** automatically and calls run server-side (keys never reach the browser). Status is shown on the **Integrations** page.

8. Local LLM Evaluation (Optional)

```
ollama pull llama3.2:1b          # any small model
```

In the app: **Model Coverage** → **Register model** with deployment **local**, endpoint `http://localhost:11434/api/generate`, model name `llama3.2:1b`, then click **Evaluate**. OpenAI-compatible runtimes (vLLM, LM Studio, LocalAI, TGI) work via their `/v1/chat/completions` or `/v1/completions` endpoint. Unreachable endpoints fall back to a simulated evaluation.

9. CI & The Security-Score Gate

CI runs type-check, build, unit + integration tests, and an **AI Security Score gate** that fails the build below a threshold:

```
cd server
SCORE_MIN=60 SCORE_PROJECT=proj-copilot npm run score:gate
```

10. Production Hardening Checklist

- Set a strong `JWT_SECRET` and `NODE_ENV=production`.
- Terminate **TLS** in front of the API (enables `Secure` cookies).
- Use managed **Postgres** with committed migrations + backups.
- Replace demo accounts; create the real Owner via `create:admin`.
- Supply vendor keys from a secret store; confirm integrations show **live**.
- Lock `CORS_ORIGIN` to your real frontend origin.
- Run `npm test` (frontend + server) and the score gate in CI.

11. Troubleshooting

SYMPTOM	FIX
Login won't accept demo creds	DB not seeded — run <code>npm run db:seed</code>
Logged out after API restart	<code>JWT_SECRET</code> not set (ephemeral secret rotated)
Header shows Local mode	Backend unreachable — start the API / check <code>CORS_ORIGIN</code>
Integration shows mock	No API key configured for that vendor (expected)
Evaluation says simulated	Model endpoint unreachable from the server
Docker API container exits	Check <code>docker compose logs api</code> (DB must be healthy first)

Part II · Usage Guide

Roles in the tags below indicate who can perform an action. **Viewers** see everything but editing/run actions are hidden or disabled. In **Local mode** (no backend) everything is enabled for a single user.

1. Sign In

1. Open the app (`localhost:5173` in dev, `localhost:8080` via Docker).
2. Enter email + password, or click a **demo account** (password `Demo!1234`).
3. You land on the **Dashboard**. Your session is private (httpOnly cookie); your role shows top-right. Sign out from the account menu.

2. Dashboard — Read Your Posture

- **Top row:** protected layers, open risks, critical gaps, active incidents.
- **Vendor Protection & Model Coverage:** files scanned, malicious blocked, injections blocked, models covered, local protected, unguarded, high-risk.
- **AI Security Score** ring with the bonuses/penalties behind it; **Layer Posture**, recent incidents, compliance status.

Use the **project selector** (top-left) to switch projects; set a default in Settings.

3. LLM Stack Map — See The Architecture

1. Open **LLM Stack Map**.
2. Pan/zoom the defense-in-depth flow; nodes are colored by status.
3. **Click any node** to open the inspector. Editors change status, owner, evidence link, notes, or delete.

4. Layer Assessment — Mark Maturity AppSec / Owner

Set each control **Not implemented** → **Planned** → **Implemented** → **Verified**. Per-layer coverage % updates instantly and the score recalculates.

5. Risk Register — Track Threats SOC / AppSec / Owner

1. Filter by severity; expand a risk to see mitigating controls + OWASP mapping.
2. Change a risk's **status** (Open → Mitigating → Mitigated / Accepted).

6. Control Library & Architecture Builder

- **Control Library** — browse/filter all controls; AppSec / Owner add a control or change status inline.
- **Architecture Builder** — drag controls from the palette onto the canvas and arrange them; click a node to edit.

7. File Security Gateway — Scan A File SOC / AppSec / Owner

1. Choose an **ingestion source** and **scan engine** (Check Point Threat Emulation / TE File Scan).
2. **Drop a file** (or browse). It's hashed locally and scanned.
3. Read the **verdict**; malicious & suspicious files are **blocked from retrieval**. Open the evidence viewer or **Retry**.

Demo tip: filenames containing `malware` , `.exe` , `macro` , or `invoice-` return bad verdicts so blocking is demonstrable.

8. Lakera Guard — Inspect A Prompt SOC / AppSec / Owner

1. Pick **Input** or **Output** stage; paste a prompt (or use a sample).
2. Click **Send to Lakera Guard** → risk score, decision (allow / block / redact / escalate), and violation categories; added to the **Inspection Log**. Tune thresholds in the Policy panel.

9. Model Coverage — Register, Test & Evaluate

1. AppSec / Owner **Register model** — provider, deployment (hosted/local/custom), endpoint, environment, sensitivity, owner, use case. Local models add network-isolation / internet-access / integrity controls.
2. Toggle guardrails (input/output/tool/logging); per-model **risk level** updates.
3. **Test** — checks the endpoint is reachable (latency).
4. **Evaluate** — runs an adversarial probe suite (prompt injection, system-prompt leakage, jailbreak, harmful content, PII) and reports a **resilience score** with each probe's verdict + the model's actual response. Unreachable endpoints produce a **simulated** result.

Evaluate a real local LLM (Ollama):

```
ollama pull llama3.2:1b
# Register a local model with endpoint http://localhost:11434/api/generate
# and model name "llama3.2:1b", then click Evaluate.
```

10. Integrations

See each vendor connector's **status** (mock / connected), auth method, base URL, and protected layers. Owner enable/disable a connector. The **API Key Configuration** panel lists the env variables to switch to live mode.

11. Compliance — Frameworks & Reports

1. Pick a framework — **OWASP LLM Top 10**, **NIST AI RMF**, **EU AI Act**, **ISO/IEC 42001**, **MITRE ATLAS**. Each shows coverage % and covered/partial/gap counts.
2. Expand a requirement to see the **controls mapped** to it and their status.
3. **Export report** downloads a per-framework Markdown compliance report.

12. Report Generator

The full assessment — executive summary, architecture overview, layer findings, control status, gaps, remediation plan. Export as **Markdown** or **PDF** (Print).

13. Settings

- **Profile & Session** — your name, email, role, and session info.
- **Preferences** — language, default project, notifications. These save to **your account** and follow you across devices/sessions.
- **User Management** Owner — **Add user** (email, name, role, temporary password); change role inline, reset password, or delete. You can't delete yourself or the last Owner.

★ Quick Role Reference

ACTION	OWNER	APPSEC	SOC	VIEWER
View everything	✓	✓	✓	✓
Edit controls / models	✓	✓	—	—
Edit risks / incidents	✓	✓	✓	—
Run scans / inspections / model evals	✓	✓	✓	—
Toggle integrations	✓	—	—	—
Manage users	✓	—	—	—